



Università della Calabria

Dipartimento di Matematica e Informatica
Corso di laurea in Informatica

GPGPU programming exam theory

STUDENTI

Elena Mastroia
Giovanni Brunetti

Anno accademico 2017/2018

Indice

1	GettingStarted	1
1.1	Introduzione	1
1.1.1	Concetti generali	1
1.1.2	La Pipeline	1
1.1.3	VertexInput	2
1.1.4	Vertex Shader	3
1.1.5	Vertex Array Object	3
1.2	Shaders	3
1.2.1	GLSL	4
1.2.2	Uniforms	4
1.3	Texture	4
1.3.1	Mipmap	5
1.4	Transformations (solo intro)	5
1.5	Coordinate Systems	5
1.5.1	Fasi di trasformazione	5
1.5.2	Proiezione Ortografica	6
1.5.3	Proiezione Prospettica	7
1.5.4	Trasformazione in Clip Space	7
1.5.5	Z-Buffer	8
1.5.6	Camera	8
1.5.7	Euler angles	8
2	Lighting	9
2.1	Color	9
2.2	Basic Lighting	9
2.2.1	Luce Ambientale	10
2.2.2	Luce Diffusa	10
2.2.3	Luce Speculare	11
2.3	Material	12
2.4	Lighting Maps	12
2.4.1	Diffuse Maps	12
2.4.2	Specular Maps	12
2.5	Light caster	12
2.5.1	Luce direzionale	12
2.5.2	Punto luce	13
2.5.3	Spotlight	14
2.6	Multiple Lights	15
3	Advanced Lighting	16
3.1	The Blinn-Phong model	16
3.2	Shadow Mapping	17
3.2.1	Sommario dei passi	19
3.2.2	Il problema dello Shadow Acne	19
3.2.3	Il problema Peter panning	20
3.2.4	Il problema delle ombre dai bordi a blocchi frastagliati	21

3.2.5	Orthographic vs Projection shadow	21
4	Bonus	22
4.1	Appunti inizio	22
4.1.1	Fase di rendering	22

Capitolo 1

GettingStarted

1.1 Introduzione

OpenGL è una libreria che fornisce un'interfaccia per la programmazione di applicazioni multiplatforma per la renderizzazione di vettori grafici 3D (solo). OpenGL è di per sé una grande macchina a stati, composta da un insieme di variabili che definisce il suo funzionamento. Ogni stato di OpenGL è detto comunemente **contesto OpenGL**. Questo stato si può semplicemente cambiare modificando alcune opzioni, buffer e, successivamente, eseguendo il rendering del contesto corrente si può visualizzare il risultato ottenuto. Quando si lavora in OpenGL, infatti, ci si imbatte in diverse funzioni di cambiamento di stato che permettono di cambiare lo stato, e altre funzioni che permettono di eseguire determinate operazioni basate sul contesto corrente.

Un oggetto in OpenGL non è altro che una raccolta di opzioni che rappresentano un sottoinsieme dello stato di OpenGL.

1.1.1 Concetti generali

In OpenGL ogni cosa è in uno spazio 3D, ma gli schermi e le finestre sono array di Pixel 2D. Dunque gran parte del lavoro svolto da OpenGL è quello di trasformare coordinate 3D in pixel 2D visibili sullo schermo. Questo processo è svolto dalla **pipeline grafica** di OpenGL. Essa può essere divisa in 2 grandi parti: una prima trasforma le coordinate 3D in pixel 2D, ed una seconda che si occupa di trasformare i pixel in pixel colorati.

Tra una coordinata 2D ed un pixel c'è una differenza: una coordinata 2D è una rappresentazione molto precisa di dove il punto è in uno spazio 2D, mentre un pixel 2D è un'approssimazione del punto stesso nel limite dello schermo (finestra visibile).

1.1.2 La Pipeline

La pipeline prende in input un insieme di coordinate 3D e le trasforma in pixel colorati 2D, mostrandoli sullo schermo. Per far ciò essa svolge determinati step, ognuno dei quali ha bisogno dell'output dello step precedente per funzionare. Ognuno di questi step sono altamente specializzati in determinate funzioni e possono essere facilmente eseguiti in parallelo.

Proprio grazie alla loro natura parallela, la maggior parte delle GPU odierne hanno migliaia di piccoli core di elaborazione per elaborare rapidamente i dati all'interno della pipeline eseguendo dei piccoli programmi per ogni step della pipeline stessa. Questi piccoli programmi sono detti **shaders**, i quali sono scritti in **OpenGL Shading Language**, (**GLSL**). La prima parte della pipeline è il **vertex shader** che prende come input un singolo vertice. L'obiettivo principale di questo step è quello di trasformare coordinate 3D in nuove coordinate 3D applicando sui vertici in input e sui loro attributi diverse trasformazioni.

La fase primaria di assemblaggio prende in input tutti i vertici dal vertex shader e forma una primitiva assemblando tutti i punti.

L'output della primitiva assemblata è passata in seguito al **geometry shader**, il quale prende in input una collezione di vertici che formano la primitiva ed ha la capacità di generare altre forme primitive, creando nuovi vertici.

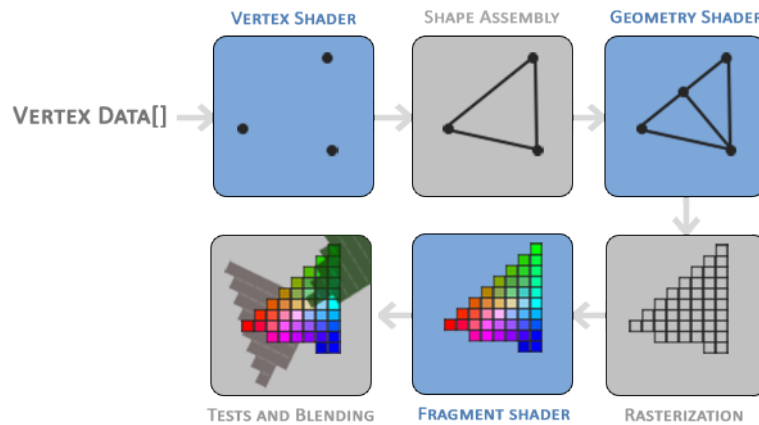


Figura 1.1: Pipeline Grafica

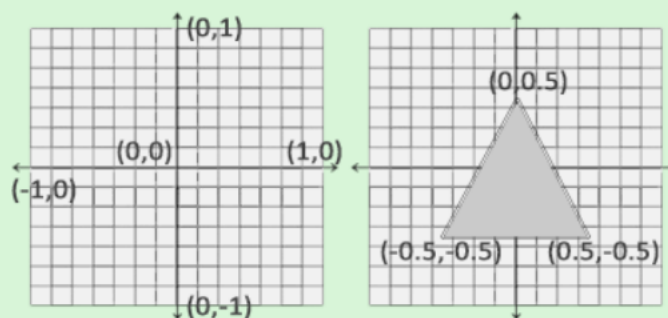
L'output del geometry shader viene in seguito passato allo step seguente, la **rasterization**, nella quale le primitive vengono mappate ai pixel corrispondenti nella finestra visiva, creando dei fragment che saranno utilizzati in seguito dai **fragment shaders**. Prima che i fragment shaders vengano eseguiti, si ha una fase di clipping, in cui tutti i frammenti fuori dalla finestra vengono scartati, in modo da aumentare le performances. Ogni fragment è l'insieme dei dati necessari ad OpenGL per renderizzare un singolo pixel!

Il principale obiettivo del fragment shader è quello di calcolare il colore finale di un pixel, fase in cui tutti gli effetti avanzati di OpenGL si verificano. Lo shader, come detto prima, contiene infatti tutti i dati per calcolare il pixel finale, quali luci, ombre, colore ecc...

1.1.3 VertexInput

Normalized Device Coordinates (NDC)

Once your vertex coordinates have been processed in the vertex shader, they should be in **normalized device coordinates** which is a small space where the x, y and z values vary from **-1.0** to **1.0**. Any coordinates that fall outside this range will be discarded/clipped and won't be visible on your screen. Below you can see the triangle we specified within normalized device coordinates (ignoring the z axis):



Unlike usual screen coordinates the positive y-axis points in the up-direction and the **(0,0)** coordinates are at the center of the graph, instead of top-left. Eventually you want all the (transformed) coordinates to end up in this coordinate space, otherwise they won't be visible.

Your NDC coordinates will then be transformed to **screen-space coordinates** via the **viewport transform** using the data you provided with `glViewport`. The resulting screen-space coordinates are then transformed to fragments as inputs to your fragment shader.

La memoria della GPU viene gestita tramite i **vertex buffer object** che memorizzano un numero elevato di vertici nella memoria della GPU. Il vantaggio dell'utilizzo dei suddetti è quello di poter inviare grandi quantità di dati contemporaneamente alla GPU senza essere obbligati ad inviare un singolo vertice per volta.

Far ciò è utilissimo in quanto il passaggio di dati verso la GPU è relativamente lento, dunque mandare il maggior numero di dati in contemporanea è di vitale importanza per le prestazioni.

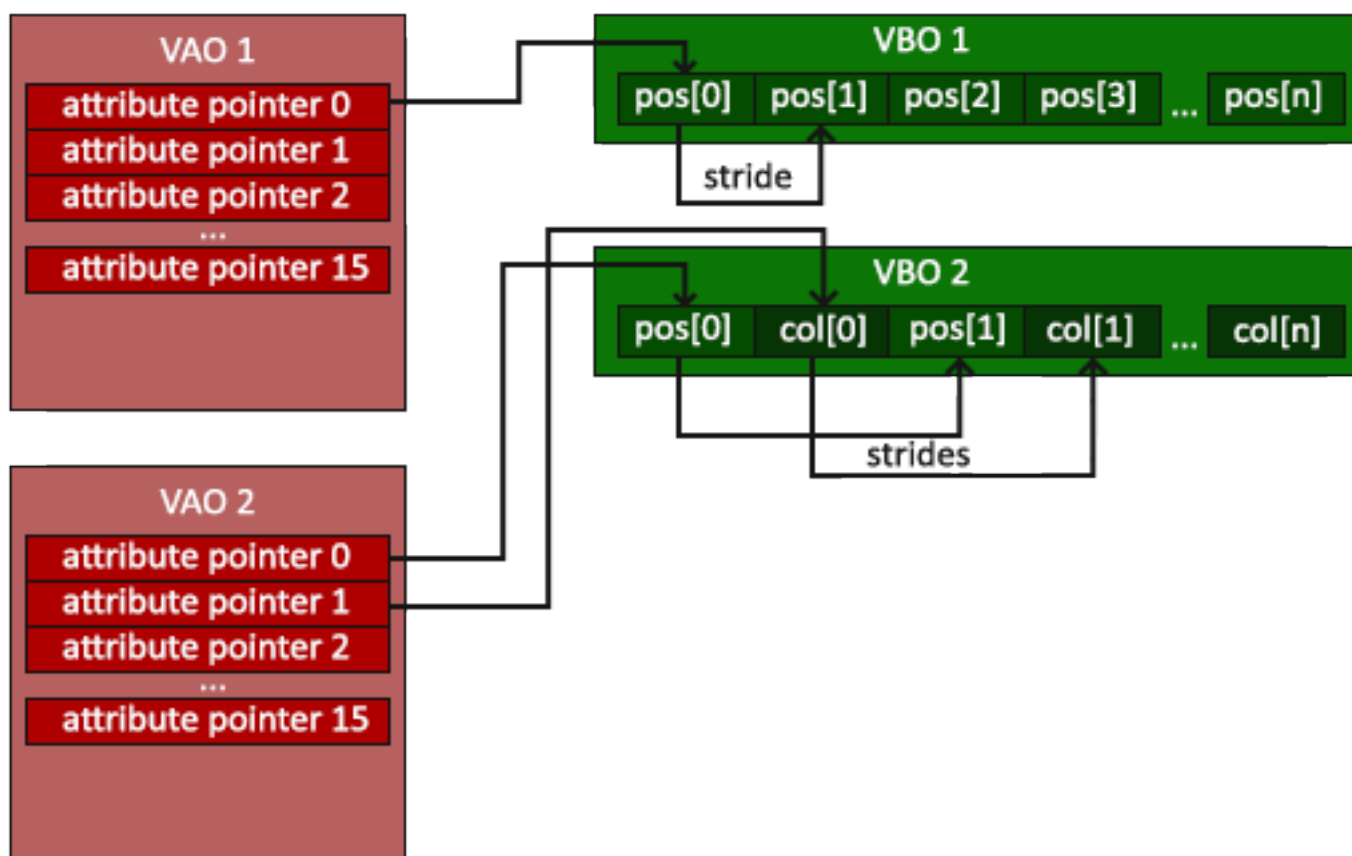
1.1.4 Vertex Shader

Vector

In graphics programming we use the mathematical concept of a vector quite often, since it neatly represents positions/directions in any space and has useful mathematical properties. A vector in GLSL has a maximum size of 4 and each of its values can be retrieved via `vec.x`, `vec.y`, `vec.z` and `vec.w` respectively where each of them represents a coordinate in space. Note that the `vec.w` component is not used as a position in space (we're dealing with 3D, not 4D) but is used for something called **perspective division**. We'll discuss vectors in much greater depth in a later tutorial.

1.1.5 Vertex Array Object

Un vertex array object, detto **VAO**, può essere associato come un vertex buffer object, in modo che qualsiasi chiamata successiva a quel vertice possa essere memorizzata all'interno del vao.



1.2 Shaders

Gli shader sono piccoli programmi che lavorano sulla GPU. Questi programmi vengono eseguiti per ciascuno degli step della pipeline grafica. In senso stretto, gli shader non sono altro che programmi che trasformano gli input in output. Essi lavorano in modo isolato, in quanto non sono autorizzati a comunicare tra di loro. L'unica comunicazione degli shader è attraverso input e output.

1.2.1 GLSL

Gli shader sono scritti in linguaggio C-like GLSL, un linguaggio personalizzato per l'uso grafico contenente utili funzioni specifiche mirate alla manipolazione di vettori e matrici.

Ogni shader può specificare input e output utilizzando le parole chiave **in** e **out**. Fondamentale è che una variabile di output corrisponda a una variabile di input del successivo livello shader, cioè, la variabile "out" del primo livello di shader deve avere lo stesso nome e tipo di una variabile "in" del secondo livello di shader. Il vertex shader deve ricevere una qualche forma di input altrimenti sarebbe inutile. Il vertex shader differisce per la tipologia del suo input, in quanto, riceve l'input direttamente dal contenitore e dal buffer che contiene trasporta i dati del vertice. Per definire come sono organizzati i dati del vertice si specificano le variabili di input con i metadati di posizione, in modo che possiamo configurare dove andare a reperire gli attributi del vertice sulla CPU (layout (posizione = 0)).

Un'altra cosa importante è che lo shader richiede una variabile di output vec4 contenente un colore, poiché i fragment shaders generano un colore di output finale, il quale, se non specificato sarà nero (o bianco).

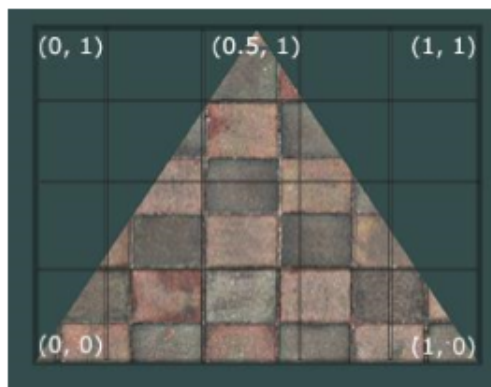
1.2.2 Uniforms

Sono un altro modo per trasferire i dati dalla CPU agli shader della GPU. Le uniforms sono leggermente diverse rispetto ai vertex shades. Prima di tutto, le uniformi sono globali (condivisi fra tutti gli shader, in secondo luogo, qualunque sia l'impostazione del valore, le uniforms manterranno i loro valori fino a quando non saranno ripristinati o aggiornati.

1.3 Texture

Una texture è un'immagine 2d (esistono anche 1d e 3d) usati per aggiungere dettagli ad un oggetto. Poiché possiamo inserire molti dettagli in una singola immagine dobbiamo cercare di dare l'illusione che l'oggetto sia estremamente dettagliato senza specificare vertici aggiuntivi.

Per mappare una texture abbiamo bisogno di associare ogni vertice di un oggetto con la parte di texture a cui dovrà corrispondere. Ogni vertice deve avere dunque una coordinata texture associata che specifica la parte d'immagine da campionare, il fragment shader poi si occuperà della restante parte. Le coordinate delle texture permettono il recupero della texture. Questo processo è detto campionamento. Le coordinate della texture vanno da (0,0), per l'angolo inferiore sinistro a (1,1), per l'angolo superiore destro dell'immagine Texture.



Ad esempio in questo triangolo le coordinate dei vertici permetteranno di selezionare solo la parte illuminata dell'immagine, ignorando la restante. Ora non ci resta che passare le coordinate al vertex shader, che a sua volta lo passa al fragment che interpola tutte le coordinate creando l'immagine per ciascun frammento.

OpenGL si occupa di mappare ogni pixel ad un texel, tuttavia non c'è sempre una corrispondenza 1 a 1 specialmente quando la risoluzione della texture è molto differente da quella dei pixel a disposizione. Quindi il suo compito è quello di decidere quale colore assegnare ad ogni pixel in base alla texture. Ci sono 2 approcci

possibili 1. Assegna ad un pixel il colore in base alla distanza più vicina fra il suo centro e la coordinata della texture (metodo di default) 2. Assegna al pixel un colore ottenuto interpolando il colore delle coordinate della texture più vicino ad esso approssimando il colore dei vari texel

1.3.1 Mipmap

Supponiamo di visualizzare un grande ambiente con tanti oggetti di piccole dimensioni sui quali sono attaccati delle texture ad alta risoluzione. In questo contesto, è facile intuire che essi in realtà essendo di piccole dimensioni producono solamente pochi fragment di conseguenza OpenGL trova delle difficoltà nel trovare il giusto colore da assegnare ad ogni fragment a causa dell'alta risoluzione della texture e questo produrrà un effetto visibile su gli oggetti in questione, per non parlare, inoltre, dello spreco di memoria che comportano l'uso di texture ad alta risoluzione in questo caso. Per risolvere questa cosa OpenGL usa il concetto di mipmaps che è in sostanza una collezione di texture ognuna delle quali è due volte più piccola di quella precedente. A questo punto, esso utilizzerà la texture che meglio si adatta alla distanza dell'oggetto. Dato che l'oggetto è lontano, la risoluzione più piccola non sarà visibile all'utente e comunque le prestazioni risultano essere migliori.

1.4 Transformations (solo intro)

Ora sappiamo come creare oggetti, colorarli e dare loro un aspetto dettagliato usando le texture, ma abbiamo ancora oggetti statici. Si potrebbe renderli dinamici cambiando i loro vertici e riconfigurando i loro buffer ad ogni frame, ma sarebbe dispendioso a livello prestazionale. Esistono modi molto migliori per trasformare un oggetto, ovvero usando **oggetti matrice**.

1.5 Coordinate Systems

La trasformazione delle coordinate in **NDC**, cioè in coordinate applicabili allo schermo, viene in genere eseguito in più fasi, in cui gli oggetti vengono trasformati in diversi sistemi di coordinate, prima di arrivare alla forma finale.

Il vantaggio di trasformarli in diversi sistemi di coordinate intermedie è che alcune operazioni sono più facili in certi sistemi rispetto che in altri. Ci sono un totale di 5 diversi sistemi importanti:

1. Local space (or Object space)
2. World Space
3. View space (or Eye space)
4. Clip space
5. Screen space

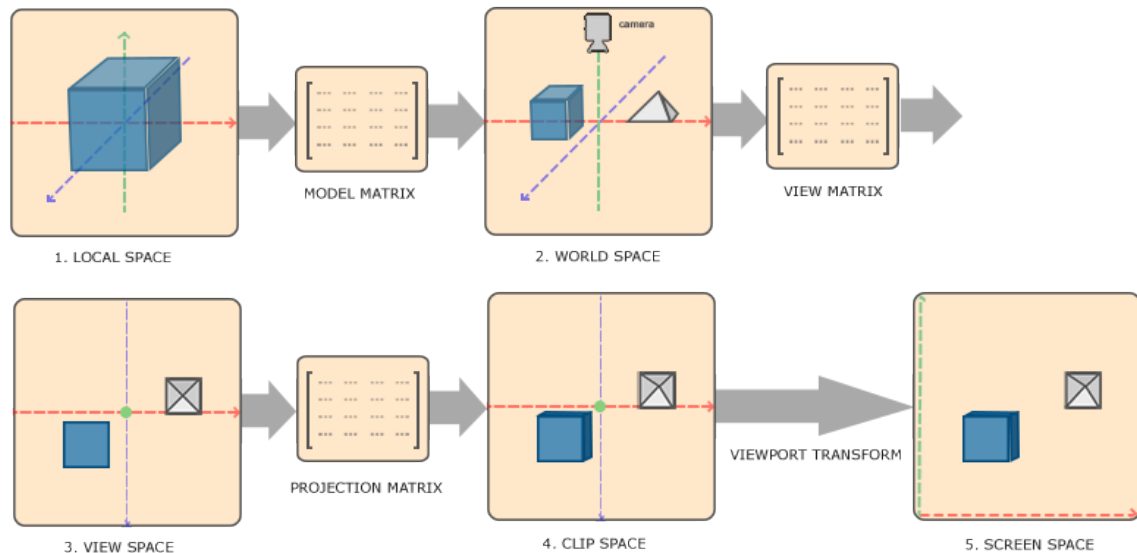
1.5.1 Fasi di trasformazione

Per trasformare le coordinate in un determinato spazio nello spazio di coordinate successivo si usano le matrici di trasformazione. Le più importanti sono il modello, la vista e la matrice di proiezione. Le coordinate del nostro vertice vengono inizialmente impostate come coordinate locali, successivamente elaborate in coordinate globali, coordinate di visualizzazione, coordinate clip e infine in coordinate dello schermo.

La trasformazione si basa su alcune fasi:

1. Le coordinate **local space** sono le coordinate in cui inizia il tuo oggetto in base alla sua origine locale.
2. Nel prossimo passo si trasformano le coordinate locali in coordinate **world space**, cioè nello spazio globale. Queste coordinate sono relative a un'origine globale, comune a tutti gli oggetti, in modo che sia possibile metterle in relazione con quelle degli altri oggetti presenti nel world space.
3. Le coordinate globali vengono trasformate in coordinate **view space**, in modo tale che ciascuna coordinata sia trasformata dal punto di vista della telecamera o dello spettatore.

4. Dopo che le coordinate sono nello view space, vengono trasformate in coordinate **clip space**, vengono elaborate nell'intervallo -1.0 e 1.0 e si determinano i vertici che saranno visualizzati sullo schermo.
5. Infine si trasformano le coordinate clip space in coordinate **screen space** attraverso un processo che trasforma le coordinate dall'intervallo -1.0 e 1.0 alla gamma di coordinate definite da glViewport. Le coordinate risultanti vengono quindi inviate alla rasterizzazione che li trasforma in frammenti.



Per trasformare le coordinate del view space al clip space si definisce una **matrice di proiezione** che specifica un intervallo di coordinate, ad es. -1000 e 1000 in ogni dimensione.

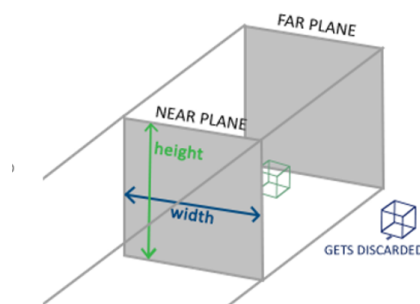
Questa finestra di visualizzazione creata dalla matrice di proiezione viene detta **frustum**, ed ogni coordinata che finisce all'interno di questo riquadro si troverà nella schermata dell'utente.

La matrice di proiezione per trasformare le coordinate view space in coordinate clip space può assumere due forme diverse, in cui ogni forma definisce il suo unico frustum. Infatti, possiamo creare una matrice di proiezione ortografica o una matrice di proiezione prospettica.

1.5.2 Proiezione Ortografica

Una matrice di proiezione ortografica definisce una scatola simile a un cubo che definisce lo spazio di ritaglio, in cui ogni vertice esterno a questo spazio è scartato. Quando si crea una matrice di proiezione ortografica, si specificano la larghezza, l'altezza e la lunghezza del frustum visibile. Tutte le coordinate che finiscono all'interno di questo frustum dopo la trasformazione in clip-space con la matrice di proiezione ortografica non verranno scartate.

Il frustum definisce le coordinate visibili ed è specificato da una larghezza, un'altezza e da due piani, uno vicino e uno lontano. Qualsiasi coordinata davanti al piano vicino viene ritagliata, lo stesso vale per le coordinate dietro il piano lontano. Il frustum ortogonale mappa direttamente tutte le coordinate all'interno del frustum attraverso coordinate del dispositivo normalizzate. Questo poiché la componente w di ciascun vettore non viene toccata; se la componente w è uguale a 1.0 la divisione prospettica non cambia le coordinate.

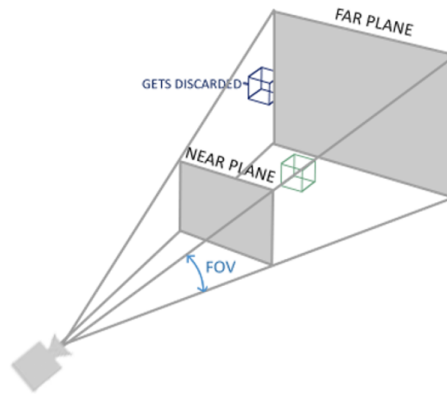


1.5.3 Proiezione Prospettica

Nella proiezione prospettica si aggiunge il concetto di distanza, ovvero si cerca di simulare la realtà, in cui un oggetto, a seconda della distanza a cui ci troviamo sembra cambiare dimensione.

La matrice di proiezione mappa un dato intervallo di frustum per ritagliare lo spazio, ma manipola anche il valore w di ogni coordinata in modo tale che quanto più lontana è una coordinata dal visualizzatore (utente), tanto più alto diventa w .

Una volta che le coordinate vengono trasformate nel clip space, sono comprese nell'intervallo da $-w$ a w (qualsiasi cosa al di fuori di questo intervallo viene tagliata). OpenGL richiede che le coordinate visibili rientrino nell'intervallo -1.0 e 1.0 come output finale del vertex shader, quindi una volta che le coordinate si trovano nel clip space, viene applicata la divisione prospettica.



Il valore **fov**, che sta per campo visivo, imposta quanto grande è lo spazio delle vista. Per una vista realistica, di solito è impostato su 45 gradi.

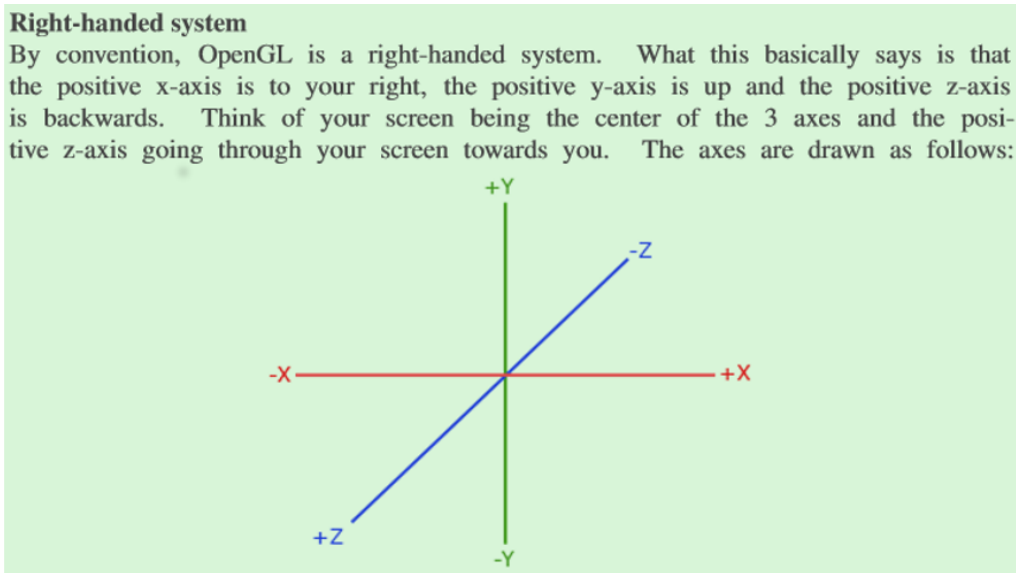
1.5.4 Trasformazione in Clip Space

Si crea dunque una matrice di trasformazione per ciascuno dei passaggi sopra indicati: modello, vista e matrice di proiezione. Una coordinata di vertici viene quindi trasformata in coordinate clip space con i seguenti passi:

$$V_clip = M_projection \cdot M_view \cdot M_model \cdot V_local$$

è da notare che l'ordine della moltiplicazione è invertito, in quanto l'ordine di moltiplicazione tra matrice è da destra a sinistra.

Spostare una telecamera all'indietro equivale a spostare l'intera scena in avanti. Questo è esattamente ciò che fa una matrice di visualizzazione, spostiamo l'intera scena in modo inverso a dove vogliamo che la telecamera si muova.



1.5.5 Z-Buffer

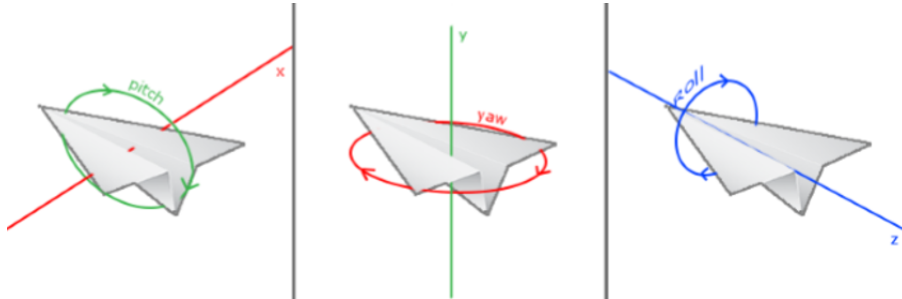
OpenGL memorizza tutte le informazioni di profondità in un buffer z, noto anche come buffer di profondità. La profondità è memorizzata all'interno di ogni frammento e ogni volta che il frammento vuole produrre il suo colore, OpenGL confronta i suoi valori di profondità con il buffer z e, se il frammento corrente è dietro l'altro frammento, viene scartato, altrimenti sovrascritto. Questo processo è chiamato test di profondità e viene eseguito automaticamente da OpenGL.

1.5.6 Camera

Per definire una telecamera abbiamo bisogno della sua posizione nel world space, della direzione in cui sta guardando, di un vettore che punta a destra e di un vettore che punta verso l'alto dalla telecamera. Si tratta effettivamente di un sistema di coordinate con 3 assi unitari perpendicolari, con la posizione della telecamera come origine.

1.5.7 Euler angles

Gli angoli di Eulero sono 3 valori che possono rappresentare qualsiasi rotazione in 3D. Ci sono 3 angoli di Eulero: beccheggio (pitch), imbardata (yaw) e rotolo (roll).



Il pitch è l'angolo che mostra quanto stiamo guardando in alto o in basso come visto nella prima immagine. La seconda immagine mostra il valore di imbardata, la quale rappresenta la magnitudine verso sinistra o destra.

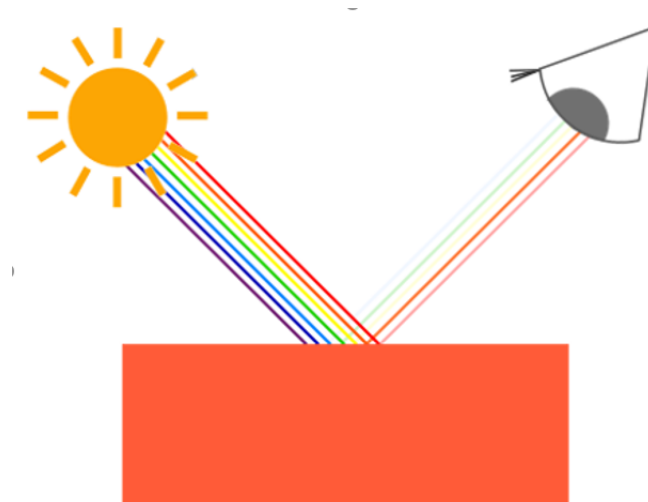
Il tiro (roll) rappresenta quanto rotoliamo, per lo più usato nelle telecamere di volo spaziale.

Capitolo 2

Lighting

2.1 Color

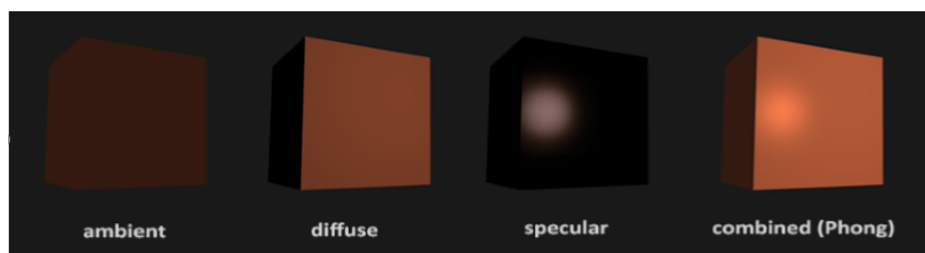
I colori sono rappresentati digitalmente usando un componente rosso, verde e blu comunemente abbreviato in **RGB**. Usando diverse combinazioni di questi 3 valori possiamo rappresentare tutti i colori conosciuti, a parte rare eccezioni. I colori che vediamo nella vita reale non sono i colori effettivi degli oggetti, ma solo i colori riflessi dall'oggetto a contatto con la luce; i colori che non sono assorbiti (rifiutati) dagli oggetti sono i colori che noi percepiamo. Queste regole di riflessione del colore si applicano direttamente nella land-graphics.



Quando definiamo una sorgente luminosa in OpenGL, si dà a questa fonte di luce un colore. In seguito se si moltiplica il colore della sorgente luminosa, con il valore del colore di un oggetto, il colore risultante è il colore riflesso dell'oggetto (e quindi il suo colore percepito).

2.2 Basic Lighting

L'illuminazione in OpenGL si basa pertanto su approssimazioni della realtà, usando modelli semplificati, con un aspetto relativamente simile alla realtà ma molto più facili da elaborare. Uno di questi modelli è il modello di illuminazione di **Phong**. I principali elementi costitutivi del modello di Phong sono 3: illuminazione ambientale, diffusa e speculare.



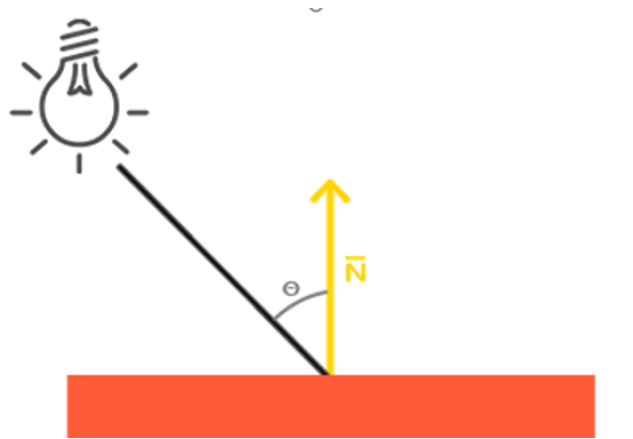
- Luce ambientale: anche quando è buio, di solito c'è sempre una fonte di luce, anche se lieve, in qualche parte del mondo (la luna, una luce lontana). Di conseguenza gli oggetti non sono quasi mai completamente al buio., per simulare ciò si utilizza una costante di illuminazione ambientale che conferisce all'oggetto sempre un po' di colore.
- Luce diffusa: simula l'impatto direzionale di una fonte di luce su un oggetto. Questo è il componente visivamente più significativo del modello di illuminazione. Più una parte di un oggetto è rivolta verso la sorgente luminosa, più diventa luminosa.
- Luce Speculare: simula il punto luminoso di una luce che appare sugli oggetti lucidi. I punti illuminati dalla luce speculare sono spesso più inclini al colore della luce stessa, rispetto al colore dell'oggetto.

2.2.1 Luce Ambientale

Di solito non c'è un'unica fonte di luce, ma molte sparse intorno a noi, anche quando non sono visibili. Una delle proprietà della luce è che può disperdersi e rimbalzare in molte direzioni raggiungendo punti che non si trovano nelle sue immediate vicinanze; la luce può quindi riflettersi su altre superfici e avere un impatto indiretto sull'illuminazione di un oggetto. Gli algoritmi che prendono in considerazione ciò sono chiamati algoritmi di illuminazione globale, ma questi sono dispendiosi o complessi. Dato che non siamo grandi fan di algoritmi complicati e costosi, inizieremo utilizzando un modello molto semplicistico di illuminazione globale, ovvero l'illuminazione ambientale. Per aggiungere illuminazione ambientale alla scena si prende il colore della luce, lo si moltiplica con un piccolo fattore ambientale costante, lo si moltiplica con il colore dell'oggetto e lo usiamo come colore del frammento.

2.2.2 Luce Diffusa

L'illuminazione ambientale di per sé non produce i risultati più interessanti, ma l'illuminazione diffusa inizia a dare un impatto visivo significativo sull'oggetto. L'illuminazione diffusa conferisce all'oggetto più luminosità quanto più i suoi frammenti sono allineati ai raggi di luce provenienti da una qualsiasi sorgente luminosa. Conoscere l'angolo col quale il raggio di luce tocca il frammento è importante. Se il raggio di luce è perpendi-



colare alla superficie, la luce ha l'impatto maggiore. Per misurare l'angolo tra il raggio di luce e il frammento si usa un vettore detto **normale** che è un vettore perpendicolare alla superficie del frammento. Più il Θ è grande (larghezza angolo), minore è l'impatto che la luce dovrebbe avere sul colore del frammento.

Il raggio di luce diretto (**lighDir**) è un vettore di direzione risultante dalla differenza tra la posizione della luce e la posizione del frammento. Per calcolare questo raggio di luce abbiamo bisogno del vettore di posizione della luce e del vettore di posizione del frammento.

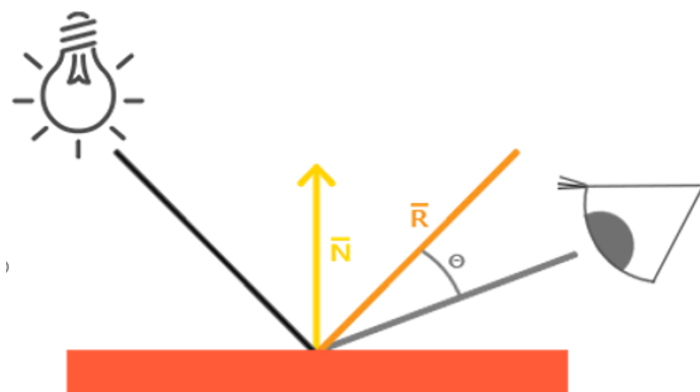
Calcolare il colore diffuso

La prima cosa che dobbiamo calcolare è il vettore di direzione tra la sorgente di luce e la posizione del frammento. Come detto prima il vettore di direzione della luce è il vettore di differenza tra il vettore di posizione della luce e il vettore di posizione del frammento.

Successivamente vogliamo calcolare l'effettivo impatto diffuso che la luce ha sul frammento corrente, prendendo il **dot product** (tipologia di moltiplicazione tra vettori) tra il vettore normale e il vettore `lightDir`. Il valore risultante viene quindi moltiplicato con il colore della luce per ottenere la componente diffusa. Maggiore è l'angolo tra entrambi i vettori e più scuro sarà il colore.

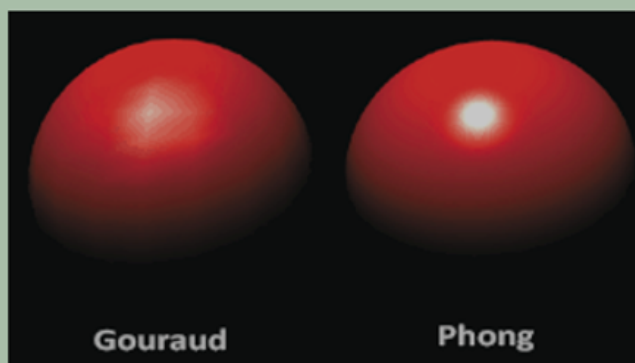
2.2.3 Luce Speculare

Proprio come l'illuminazione diffusa, l'illuminazione speculare si basa sul vettore di direzione della luce e sui vettori normali dell'oggetto, ma si basa anche sulla direzione della visuale, ad esempio la direzione con la quale il giocatore guarda il frammento. L'illuminazione speculare si basa sulle proprietà della luce di riflettersi sugli oggetti. Se pensiamo alla superficie dell'oggetto come a uno specchio, l'illuminazione speculare è la più forte poichè la luce riflessa sulla superficie sarà pressochè uguale alla luce reale. Si calcola



un vettore di riflessione riflettendo la direzione della luce attorno al vettore normale. In seguito si calcola la distanza angolare tra questo vettore di riflessione e la direzione della visuale. Più piccolo è l'angolo tra di loro, maggiore è l'impatto della luce speculare. L'effetto risultante è che vediamo un po' di luce quando guardiamo la direzione della luce riflessa attraverso l'oggetto.

In the earlier days of lighting shaders, developers used to implement the Phong lighting model in the vertex shader. The advantage of doing lighting in the vertex shader is that it is a lot more efficient since there are generally a lot less vertices than fragments, so the (expensive) lighting calculations are done less frequently. However, the resulting color value in the vertex shader is the resulting lighting color of that vertex only and the color values of the surrounding fragments are then the result of interpolated lighting colors. The result was that the lighting was not very realistic unless large amounts of vertices were used:



When the Phong lighting model is implemented in the vertex shader it is called **Gouraud shading** instead of **Phong shading**. Note that due to the interpolation the lighting looks a bit off. The Phong shading gives much smoother lighting results.

2.3 Material

Nel mondo reale, ogni oggetto reagisce in modo diverso alla luce. Ad esempio, gli oggetti in acciaio sono più lucidi di quelli in argilla o di legno. Ogni oggetto risponde inoltre in modo diverso alle luci speculari. Alcuni oggetti riflettono la luce senza troppa dispersione, risultando in piccoli punti salienti mentre altri con molta dispersione creano un raggio più ampio. Per simulare questo effetto in OpenGL bisogna specificare i tipi di materiali.

Quando descriviamo gli oggetti, possiamo definire un colore per ciascuno dei 3 componenti di illuminazione: ambientale, diffusa e speculare. Specificando un colore per ciascuno dei componenti, abbiamo un controllo a **fine-grained** sull'output del colore.

Nello shader di frammenti si crea una struttura per memorizzare le proprietà del materiale, si definisce un vettore che descrive un colore per ogni elemento del modello di Phong. Il campo del materiale ambientale definisce il colore l'oggetto riflette nell'illuminazione ambientale, il quale, di solito è uguale al colore dell'oggetto stesso. Il campo del materiale diffuso definisce il colore dell'oggetto sotto illuminazione diffusa, questo colore è impostato sul colore desiderato per l'oggetto. Il campo speculare invece determina l'impatto cromatico che una luce speculare ha sull'oggetto.

2.4 Lighting Maps

Ogni oggetto ha la possibilità di avere un proprio materiale che reagisce in modo diverso alla luce. Questo è fondamentale per dare ad ogni oggetto un aspetto proprio in una scena illuminata, ma non offre ancora troppa flessibilità sull'output visivo dell'oggetto. Gli oggetti nel mondo reale di solito non sono costituiti da un singolo materiale, ma sono costituiti da diversi materiali.

Per dare un'aspetto più realistico al nostro mondo ci sono le mappe diffuse e speculari, le quali permettono di influenzare la diffusione e la componente speculare di un oggetto con molta più precisione.

2.4.1 Diffuse Maps

In base alla posizione del frammento sull'oggetto è possibile recuperare un vettore di colore. Questo procedimento è molto simile a quello usato per le textures. La differenza è solo il nome che viene utilizzato per lo stesso principio di base: utilizzare un'immagine per wrappare un oggetto, il quale viene indicizzato per vettori di colore univoci per ogni frammento. Nelle scene illuminate questo è chiamato diffuse maps poiché un'immagine texture rappresenta tutti i colori diffusi dell'oggetto.

2.4.2 Specular Maps

Per controllare il modo in cui ogni parte dell'oggetto rifletta la luce speculare, ognuna con intensità diversa, si usano le specular maps. Ci troviamo di fronte a un discorso simile alle diffuse maps, si può usare anche una mappatura texture solo per le luci speculari. Il tutto si riduce nel generare una trama in bianco e nero (anche a colori volendo) che definisce l'intensità con la quale ogni parte d'oggetto si rapporta alla luce speculare.

2.5 Light caster

Nella realtà noi non abbiamo una sola sorgente di luce che è un singolo punto nello spazio bensì abbiamo diversi tipi di luce che agiscono tutte in modo differente. Una sorgente di luce che getta luce su gli oggetti è chiamata **light caster**.

2.5.1 Luce direzionale

Quando una sorgente di luce è lontana, i raggi di luce che provengono da essa sono vicini e paralleli fra di loro. Essi sembrano provenire tutti dalla stessa direzione indipendentemente da dove l'oggetto e/o l'osservatore si trovi. Quando una sorgente di luce è modellata in modo da essere infinitamente lontana è chiamata **directional light** poiché tutti i suoi raggi hanno la stessa direzione; essa, è indipendente dalla posizione della sorgente di luce.

Un giusto esempio di luce direzionale è il sole. Esso non è infinitamente lontano per noi ma è così lontano

da farci percepire la sua presenza come infinitamente lontano durante il calcolo dell'illuminazione. Infatti, tutti i raggi provenienti dal sole sono modellati come paralleli.

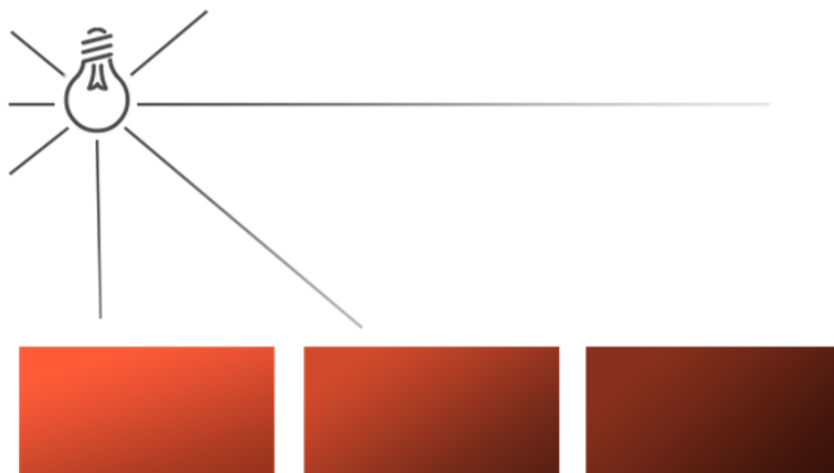
Dato che tutti i raggi di luce sono paralleli non è possibile definire come ogni oggetto si rapporta alla



posizione della sorgente di luce poiché la direzione della luce è sempre la stessa per ogni oggetto presente nella scena.

2.5.2 Punto luce

Le luci direzionali sono ottime per simulare le luci globali che illuminano l'intera scena ma al di là di essa noi vorremmo anche simulare diversi punti di luce sparpagliati all'interno della scena. Un punto di luce è una sorgente di luce con una determinata posizione all'interno di un mondo che illumina in tutte le direzioni finché i raggi di luce non svaniscono a causa della distanza. Per capire la sua caratteristica, basta pensare ad una lampadina accesa.



Attenuazione

Il fenomeno che consente di ridurre l'intensità della luce, lungo la distanza sul quale lavora un raggio di luce, è generalmente chiamato **attenuazione**. Un modo per ridurre l'intensità in base alla distanza è quello di utilizzare semplicemente una equazione lineare la quale si occuperà di questo e farà risultare gli oggetti più distanti meno illuminati.

Tuttavia, qualche equazione lineare tende un po' a sbagliare. Nel mondo reale, le luci sono generalmente abbastanza luminose nelle vicinanze ma la brillantezza della sorgente di luce diminuisce velocemente all'inizio mentre successivamente l'intensità rimanente diminuisce molto più lentamente lungo la distanza. Perciò

abbiamo bisogno di una formula differente per effettuare la riduzione. La formula

$$F_{att} = \frac{1.0}{K_c + K_l * d + K_q * d^2}$$

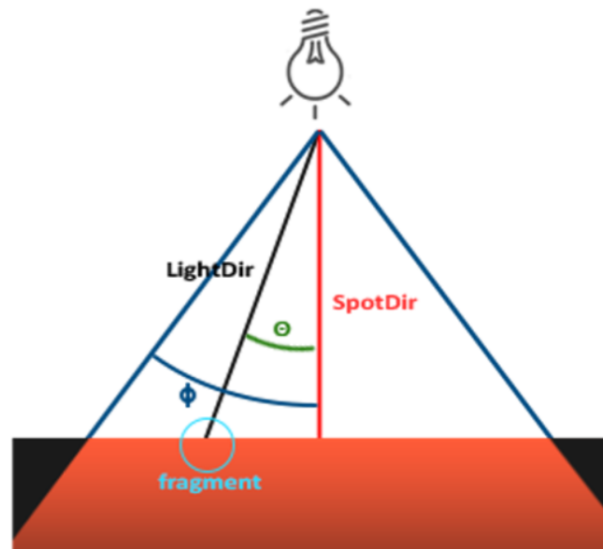
calcola un valore di attenuazione, basato sulla distanza del fragment dalla sorgente di luce, il quale sarà poi moltiplicato per il vettore di intensità della luce. Nella formula d rappresenta la distanza dal fragment alla sorgente di luce. Quindi per calcolare l'attenuazione vengono definiti 3 termini (configurabili): un termine costante K_c , un termine lineare K_l e un termine quadratico K_q

- Il termine costante è solitamente 1.0 che permette di mantenere il risultante denominatore sempre più grande di 1 poiché altrimenti incrementerebbe l'intensità arrivati ad una determinata distanza e questo non è l'effetto che stiamo cercando.
- Il termine lineare è moltiplicato per il valore della distanza che riduce l'intensità in modo lineare
- Il termine quadratico è moltiplicato con il quadrato della distanza e porta ad un decremento quadratico di intensità per la sorgente di luce. Il termine quadratico sarà meno significativo comparato al termine lineare quando la distanza è piccola ma ha un'importanza molto più significativa del termine lineare quando la distanza aumenta.

2.5.3 Spotlight

Spotlight è una fonte di luce presente all'interno dell'ambiente che invece di spargere i raggi di luce in tutte le direzioni, li dirige in una sola direzione specifica. Il risultato è che solo gli oggetti all'interno del raggio formato dalla direzione vengono illuminati, mentre gli altri rimangono oscurati. Per capire perfettamente il funzionamento di una spotlight basta pensare ad un lampione o una torcia elettrica.

Uno spotlight in OpenGL è rappresentato da una posizione nel world space, una direzione e un angolo di taglio che specifica il raggio dello spotlight. Per ogni frammento si calcola se il frammento si trova tra la direzione di taglio dello spotlight e, in tal caso, si illumina il frammento di conseguenza.



- **LightDir**: il vettore che punta dal frammento alla fonte di luce.
- **SpotDir**: la direzione verso cui lo spotlight punta.
- **Phi Φ**: l'angolo di taglio che specifica il raggio dello spotlight. Tutto al di fuori di quest'angolo non è illuminato dallo spotlight.
- **Theta Θ**: l'angolo tra il vettore LightDir e il vettore SpotDir. Il valore di Θ deve essere inferiore al valore di Φ per essere all'interno dello spotlight.

Quindi, ciò che fondamentalmente dobbiamo fare, è calcolare il dot product (il coseno dell'angolo tra due vettori unitari) tra il vettore LightDir e il vettore SpotDir e confrontarlo con l'angolo di **cutoff**.

Flashlight

Una flashlight è uno spotlight situato nella posizione dello spettatore, di solito puntato dritto dalla prospettiva del giocatore. Fondamentalmente, una flashlight è uno spotlight normale, ma con la sua posizione e direzione continuamente aggiornate in base alla posizione e all'orientamento del giocatore.

Smooth/Soft edges

Per creare l'effetto di uno spotlight senza spigoli si simula uno spotlight con un cono interno e uno esterno. Si imposta il cono interno come il cono definito nella sezione precedente, ma si imposta anche un valore che offusca gradualmente la luce dall'interno ai bordi del cono esterno.

Per creare il cono esterno, si definisce semplicemente un altro valore del coseno che rappresenta l'angolo tra il vettore di direzione dello spotlight e il vettore del cono esterno (uguale al suo raggio). Ad esempio, se un frammento si trova tra il cono interno e il cono esterno, dovrebbe calcolare un valore di intensità compreso tra 0,0 e 1,0. Se il frammento si trova all'interno del cono interno, la sua intensità è uguale a 1,0 e 0,0 se il frammento si trova all'esterno del cono esterno.

Possiamo calcolare tale valore usando la seguente formula:

$$I = \frac{\Theta + \gamma}{\epsilon}$$

dove ϵ è il coseno della differenza tra il cono interno (Φ) e il cono esterno (Γ), dunque $\epsilon = \Phi - \gamma$. Il risultato I è l'intensità dello spotlight sul frammento.

2.6 Multiple Lights

Per utilizzare più di una fonte di luce nella scena si incapsulano i calcoli di illuminazione in funzioni GLSL. Il motivo è che il codice diventa rapidamente fastidioso quando vogliamo eseguire calcoli di illuminazione con più luci, con ogni tipo di luce che richiede calcoli diversi.

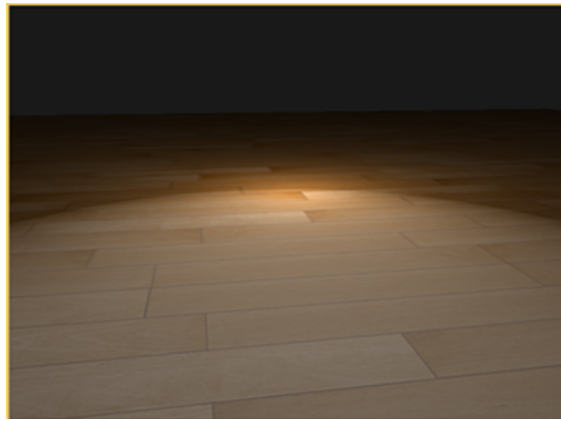
Quando si usano più luci in una scena, l'approccio è solitamente il seguente: abbiamo un singolo vettore di colore che rappresenta il colore di uscita del frammento. Per ogni luce, il colore di contributo risultante viene aggiunto al vettore del colore di uscita del frammento. Quindi ogni luce nella scena calcolerà il suo impatto individuale sul frammento e contribuirà al colore finale nell'output. Se per esempio due sorgenti di luce sono vicine al frammento, il loro contributo combinato darebbe luogo a un frammento più illuminato del frammento che verrebbe illuminato da un'unica fonte di luce.

Capitolo 3

Advanced Lighting

3.1 The Blinn-Phong model

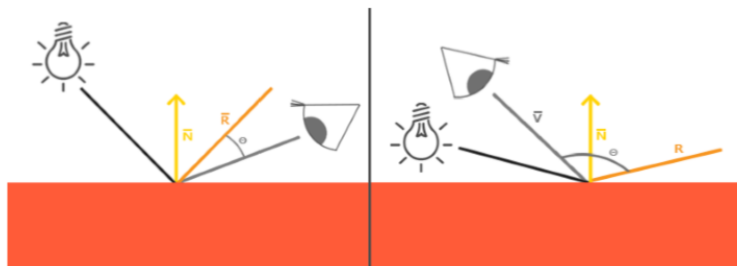
Il modello di Phong è un efficiente ed ottima approssimazione dell'illuminazione ma la sua riflessione speculare presenta evidenti problemi in alcune condizioni; in particolare, quando la proprietà della lucentezza è bassa si ottiene una grande (grezza) area speculare. La seguente immagine mostra cosa succede quando viene utilizzata un esponente di lucentezza speculare di 1.0 su un piano.



Come è possibile vedere, ai bordi l'area speculare viene immediatamente interrotta in modo grossolano. Questo è dovuto al fatto che l'angolo fra il *view vector* e il *reflection vector* non può superare i 90° .

Infatti, quando questo avviene il risultato del *dot product* fra i due vettori diviene negativo e questo porta ad un esponente speculare di 0.0.

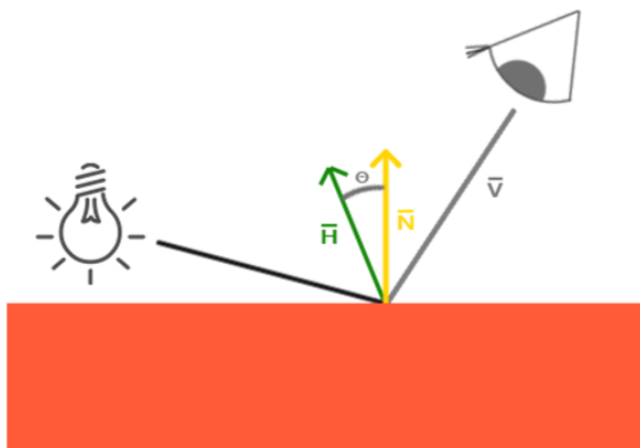
E' bene precisare che ciò avviene solamente per la componente *diffusa* dove un angolo maggiore di 90° fra il normale e la sorgente di luce significa che quest'ultima si trova al di sotto della superficie illuminata e quindi il contributo della luce diffusa dovrebbe essere pari a zero. Tuttavia, con l'illuminazione speculare noi non misuriamo l'angolo fra la sorgente di luce e il normale bensì fra i vettori *view* e *reflection direction*.



Per questa ragione è stato introdotto il modello di ombreggiatura di **Blinn-Phong** come estensione di quello di Phong.

Il modello di Blinn-Phong è molto simile ma come possiamo intuire l'approccio al modello speculare è leggermente differente cosa che consente di eliminare il problema precedentemente discusso.

Invece di fare affidamento sul vettore di riflessione verrà utilizzato il vettore chiamato *halfway* che è un vettore esattamente a metà fra la view direction e la light direction. Praticamente più questo vettore si allinea col normale della superficie illuminata maggiore sarà il contributo della componente speculare.



Quando la view direction è perfettamente allineata col (adesso immaginario) vettore di riflessione, il vettore di mezzo si allinea perfettamente col normale. Quindi, più l'osservatore si avvicina alla direzione di riflessione originale, più diventa forte ed evidente la componente speculare diventa forte.

Nella figura è possibile vedere da quale direzione l'osservatore sta guardando, l'angolo fra il vettore *halfway* e il normale della superficie non supera **mai**(.) i 90 gradi (ovviamente nemmeno quando la luce è molto vicina alla superficie). Questo produce dei risultati leggermente differenti comparati col modello di riflessione di Phong ma visualmente più plausibili rispetto a quest'ultimo; specialmente con un esponente speculare molto basso. Il modello per le ombre di Blinn-Phong è anche l'esatto modello di ombreggiatura usato nella precedente funzione della pipeline di OpenGL.

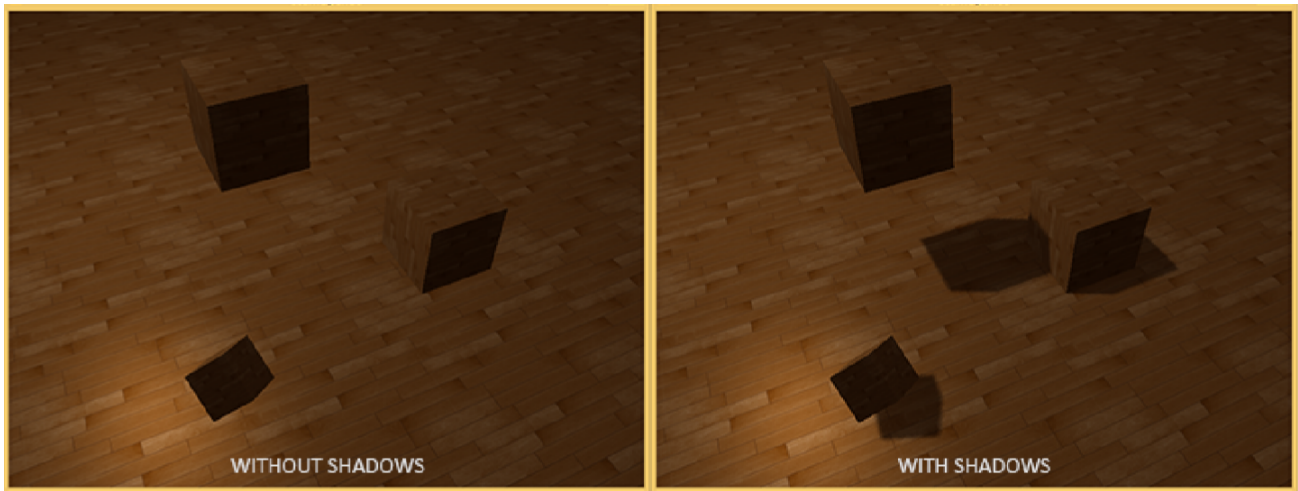
Trovare il vettore *halfway* è abbastanza semplice, basta sommare il vettore di direzione della luce con il vettore *view* e successivamente normalizzare il risultato: $\vec{H} = \frac{\vec{L} + \vec{V}}{\|\vec{L} + \vec{V}\|}$



3.2 Shadow Mapping

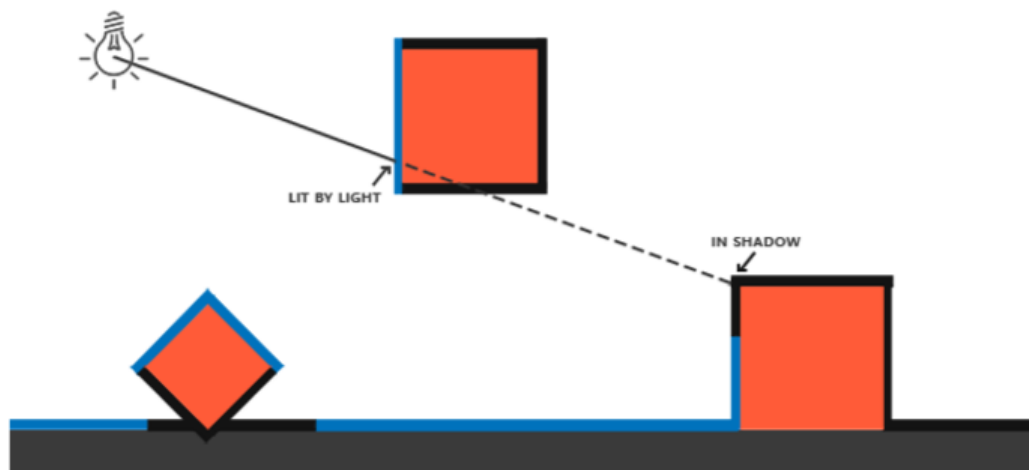
Le ombre sono il risultato dell'assenza di luce dovuta ad un'occlusione; quando il raggio di luce proveniente da una sorgente non tocca un oggetto perché viene coperto da un altro, il quale lo precede lungo la traiettoria del raggio, esso risulta essere in ombra. Le ombre aggiungono un enorme realismo alla scena illuminata e rende semplice all'osservatore capire la relazioni fra gli oggetti presenti nello spazio osservato. Esse, infatti, danno un grande senso della profondità alla scena e agli oggetti.

Ci sono numerose buone tecniche di approssimazione delle ombre. Una delle tecniche, utilizzata dalla maggior parte dei videogames, che fornisce un decente risultato e che è relativamente semplice da implementare



è la *shadow mapping*. Essa, non è molto complicata da capire, non costa molto a livello di performance ed è facilmente estensibile in algoritmi più avanzati.

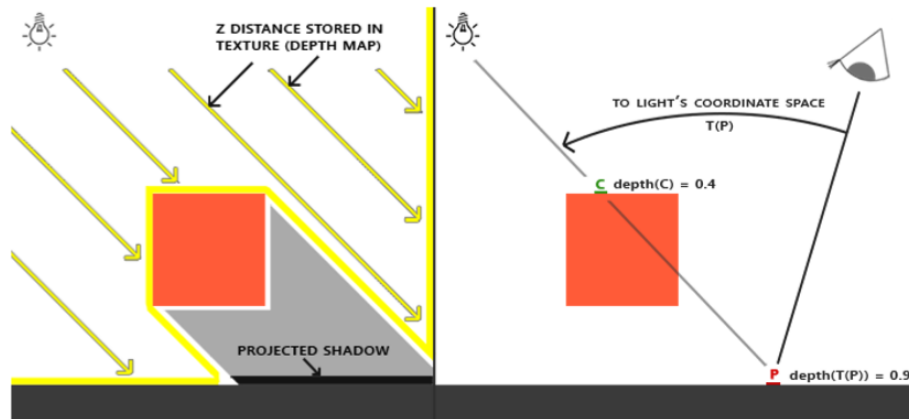
L'idea di base dello shadow mapping è abbastanza semplice: renderizzare la scena dal punto di vista della luce e qualsiasi cosa che viene vista da questa prospettiva è illuminata, tutto il resto che non è visibile deve essere invece in ombra.



Ipotizziamo che venga tracciata una linea corrispondente ad un raggio che va dalla sorgente di luce al fragment. Prima vogliamo ottenere i punti su di essa che intercettano un oggetto e successivamente confrontare il punto più vicino alla sorgente di luce con gli altri punti su questa linea. Quindi, in pratica, per ogni punto P_T trovato, viene effettuato un semplice test per vedere se esso è più in basso del punto P_V che attualmente è più vicino alla sorgente; se ciò avviene allora P_T deve essere considerato in ombra.

Tuttavia, iterare e castare in linee tutti i possibili raggi provenienti da una o più sorgenti di luce è ovviamente un approccio estremamente inefficiente e non si presta benissimo al rendering in tempo reale. E' possibile però fare qualcosa di molto simile senza però effettuare il casting del raggio utilizzando invece il **depth buffer**. Un valore all'interno di questo buffer corrisponde alla profondità di un fragment all'interno dell'intervallo $[0,1]$ dal punto di vista della camera. Alla fine di tutto, il valore di profondità mostra il primo fragment visibile dalla prospettiva della sorgente di luce. Tutte queste informazioni sulla profondità vengono memorizzate all'interno di una texture chiamata **depth map** oppure **shadow map**. Questo punto più vicino che viene utilizzato successivamente per determinare quando un fragment è in ombra.

La depth map viene creata quindi attraverso il rendering della scena dalla prospettiva della sorgente di luce utilizzando una matrice *view* e *projection* specifiche per essa. Queste due matrici insieme formano una trasformazione T che trasforma qualsiasi posizione $3D$ nelle coordinate *space* visibili dalla sorgente.



Noi renderizziamo un frammento al punto $\bar{P}$ per il quale abbiamo determinato se esso è in ombra. Per fare ciò, prima il punto $\bar{P}$ viene trasformato nelle coordinate *space* utilizzando T . Esso ora è visto dalla prospettiva della sorgente e la sua coordinata z corrisponde ovviamente alla sua profondità. Utilizzando tale punto noi possiamo anche indicizzare la depth map per ottenere la profondità più vicina dalla sorgente utilizzando il punto $\bar{C}$. finché l'indicizzazione della mappa restituisce una profondità minore della profondità al punto $\bar{P}$ è possibile concludere che esso è occluso e quindi in ombra.

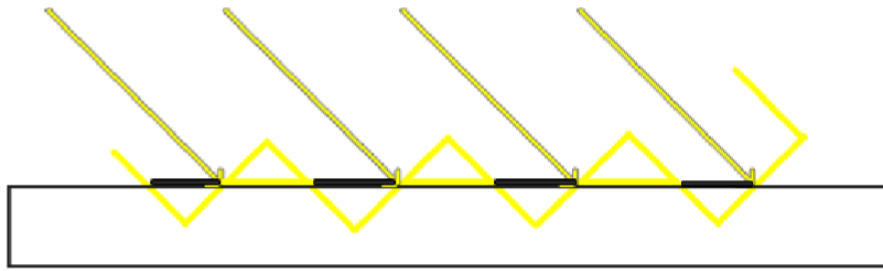
Ricapitolando, la **shadow mapping** consiste in due passaggi: prima di tutto viene renderizzata la depth map e successivamente viene effettuato il rendering della scena come avviene normalmente utilizzando in più la depth map, precedentemente creata, per calcolare quando un frammento è in ombra.

3.2.1 Sommario dei passi

- **La depth map:** Come detto prima, il primo passo richiede di generare una depth map che altro non è che la depth texture renderizzata dalla prospettiva della sorgente di luce la quale sarà utilizzata per calcolare le ombre.
- **trasformazione nel *Light Space*:** vengono utilizzate la view matrix e la projection per effettuare il render della scena dal punto di vista della sorgente di luce. Quando si modellano delle sorgenti di luce direzionali tutti i raggi sono paralleli e perciò è preferibile utilizzare una matrice di proiezione ortografica piuttosto che prospettica poiché quest'ultima applica delle deformazioni alla scena tipiche della prospettiva.
- **Render della depth map:** quando renderizziamo la scena dalla prospettiva della sorgente di luce è preferibile usare un semplice shader che trasforma solo i vertici nel light space e non molto altro.
- **Rendering delle ombre:** con un'appropriata depth map generata è possibile iniziare a generare le ombre attuali. Il fragment shader che sarà utilizzato per renderizzare la scena utilizza il modello di luce Blinn-Phong. Nello shader sarà calcolato il valore di ombra che sarà 1.0 quando il fragment è in ombra mentre 0.0 quando esso è esposto alla luce. I risultanti colori, diffuso e speculare, sono moltiplicati per questa componente di shadow. Dato che le ombre sono raramente completamente nere anche a causa delle varie dispersioni delle luci presenti nell'ambiente è bene tenere fuori dalla moltiplicazione della shadow l'ambient color.

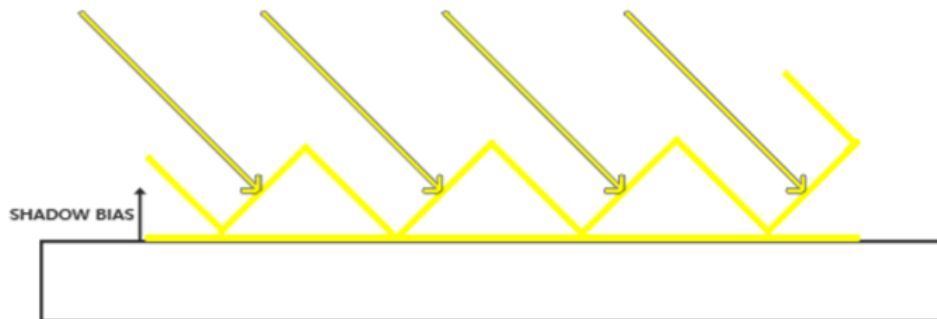
3.2.2 Il problema dello Shadow Acne

Spesso può capitare di vedere delle grandi parti di figure che dopo il calcolo della ombra presentano delle linee nere in modo alternato. Questo problema è noto come **shadow acne**. La shadow map essendo in realtà una texture ha una risoluzione limitata, definita durante l'inizializzazione della texture stessa. Di conseguenza, più frammenti possono avere facilmente lo stesso valore nella depth map quando essi sono relativamente vicini ai raggi della sorgente di luce. Anche se questo generalmente non causa grossi disagi diventa un problema quando la sorgente di luce guarda ad angolo verso la superficie (e non in modo diretto) in quanto in questo



caso anche la mappa viene creata ad angolo. Più fragments accedono quindi allo stesso *depth texel* piegato ottenendo così una discrepanza della texture.

E' possibile risolvere questo problema con un piccolo truccetto: un "taglio" chiamato **shadow bias** dove semplicemente viene spostata la profondità della superficie (o della shadow map) per un piccolo bias tale per cui i fragment non sono più erroneamente considerati sotto la superficie. Con l'applicazione del bias tutti i campioni restituiscono una profondità minore della profondità della superficie e quindi l'intera superficie è correttamente illuminata senza ombra.

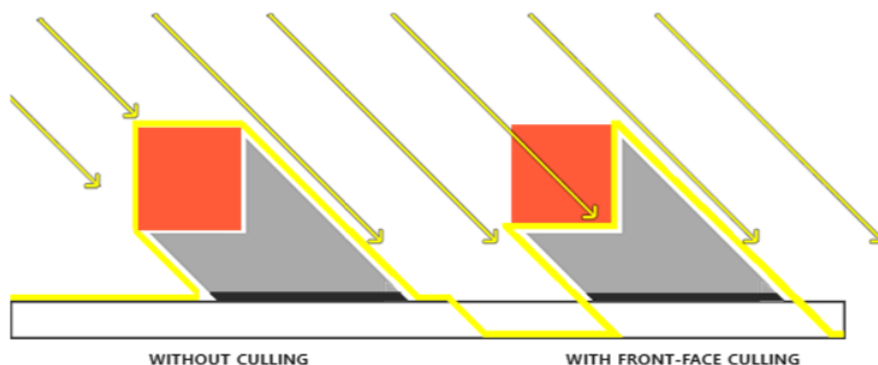


3.2.3 Il problema Peter panning

Uno svantaggio dell'usare uno shadow bias sta nel fatto che si sta applicando un offset dell'attuale profondità degli oggetti. Come risultato il bias potrebbe diventare molto grande abbastanza da rendere visibile questo offset quando si mette in relazione l'attuale posizione di un oggetto rispetto alla sua ombra.

Questo problema è chiamato **Peter Panning** poiché gli oggetti sembrano staccati dalla propria ombra. Anche in questo caso è possibile utilizzare qualche truccetto per risolvere la maggior parte dei problemi Peter Panning utilizzando un *front face culling* (abbattimento frontale, in pratica non viene considerata la superficie frontale bensì quella di dietro) quando viene effettuato il rendering della depth map.

Dato che noi abbiamo bisogno solamente del valore di profondità secondo la depth map, dovrebbe essere indifferente, per un oggetto solido, utilizzare la loro faccia frontale piuttosto che quella dietro e viceversa. Utilizzando la profondità della loro back face non si ottiene un risultato sbagliato come non importa se vengono create le ombre all'interno dell'oggetto stesso essendo che non sarebbero comunque visibili.



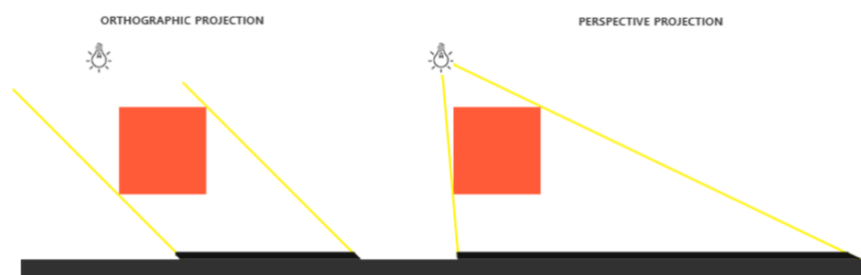
Questo effettivamente risolve il problema; tuttavia, lo fa solamente per gli oggetti solidi. Per esempi, o in un'ipotetica scena, questo funzionerebbe bene con un cubo ma non funzionerebbe con un piano per il quale ignorare la sua front face significherebbe eliminarlo completamente dall'equazione e non produrrebbe nessuna ombra. Quindi è bene utilizzare questo trucco solamente quando ha senso.

3.2.4 Il problema delle ombre dai bordi a blocchi frastagliati

Dato che la depth map ha una risoluzione fissata, spesso la profondità si spanna su più di un fragment per texel. Come risultato di questo si ottiene che più fragment campionano lo stesso valore dalla depth map e raggiungono la stessa tipologia di ombra di conseguenza è possibile notare dei bordi a blocchi frastagliati. E' possibile ridurre queste ombre a blocchi incrementando la risoluzione della depth map oppure cercando di adattare il frustum della luce il più possibile vicino alla scena.

Un'altra (parziale) soluzione a questo problema è chiamato **PCF** o *percentage-closer filtering*, il quale è un termine che fa riferimento a molte differenti funzioni di filtraggio che producono delle ombre più soft, facendole apparire meno rudi o a blocchi. L'idea è quella di campionare più di un texel dalla depth map, ogni volta con coordinate della texture leggermente differenti. Per ogni campione individuale viene controllato quando esso è in ombra e quando non lo è. Tutti questi sotto-risultati sono successivamente combinati, effettuata una "media su di essi "e così si ottiene un'ombra carina e soft.

3.2.5 Orthographic vs Projection shadow



C'è una notevole differenza fra effettuare il rendering della depth map con una proiezione ortografica piuttosto che con una prospettica. Una matrice di proiezione ortografica non deforma la scena con la prospettiva, perciò tutte le view e raggi luce sono paralleli il che la rende un'ottima matrice di proiezione per le luci direzionali. Una matrice di proiezione prospettica tuttavia deforma tutti i vertici in base alla prospettiva fornendo di conseguenza un risultato differente. Infatti, essa, ha più senso per le sorgenti di luce le quali hanno attualmente delle posizioni differenti da quelle direzionali.

Quindi, la proiezione prospettica è utilizzata per spotlights e punti luce, mentre la proiezione ortografica è utilizzata per le luci direzionali.

Un'altra sottile differenza quando si utilizza una proiezione prospettica è, che visualizzando il depth buffer, spesso si otterrà un risultato quasi completamente bianco. Questo accade perché con questa proiezione la profondità è trasformata in dei valori non lineari con la maggior parte dei suoi range vicini al *near plane*. Quindi per essere in grado di visualizzare nel modo appropriato i valori di profondità, come viene fatto per la proiezione ortografica, bisogna prima trasformare i valori di profondità non lineari in lineari.

Capitolo 4

Bonus

4.1 Appunti inizio

Sviluppando un'applicazione open core profile in realtà si sviluppano due applicazioni:

- La prima gira sull'host interno (CPU), qui vi è un compilatore che esegue l'app device e ne controlla l'esecuzione.
- La seconda gira sulla GPU, l'applicazione è scritta in linguaggio GLSL. Questa app segue un linguaggio logico detto graphics pipeline, che ricevendo qualche dato in input li processa per ottenere l'immagine desiderata.

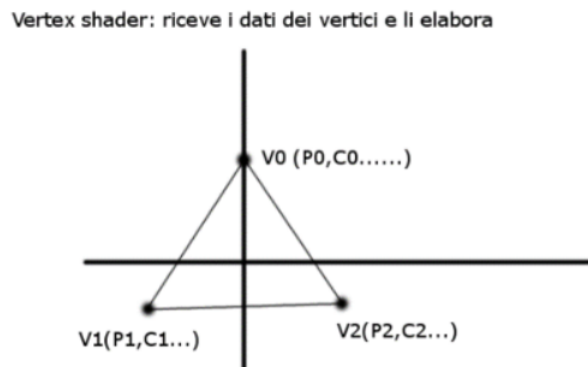
Mentre la 1 è un'app seriale la seconda è un'app **massive parallel application**, poiché gira sulla gpu che è un **massive parallel device**.

4.1.1 Fase di rendering

Ogni fase della pipeline è fatta in modo parallelo, generalmente sono piccoli ma paralleli. Le fasi più importanti sono quelle di shading, poiché esse sono il requisito minimo per effettuare qualsiasi operazione di rendering. L'informazione più semplice che possiamo dare allo shader è la posizione. Il ruolo del vertex shader è quello di processare queste informazioni. Noi utilizziamo queste informazioni per effettuare una trasformazione su tali vertici, come ad esempio una traslation. Differenze istanze di questo vertex shader vengono eseguite in parallelo. Nell'esempio dell'immagine in alto, uno si occupa di elaborare v_0 , uno v_1 e uno v_2 . Qui si lavora in spazi vettoriali e non in pixel, nonostante ciò, alla fine l'immagine dovrà essere convertita poiché il nostro shader lavora in pixel e non in vettori. Di questa fase si occupa lo stage della rasterization.

Il fragment shader assegna un colore ad ogni fragment. Abbiamo così una cosiddetta **computational explosion**.

Questo parallelismo è trasparente all'utente, egli si occupa solamente di scrivere un semplice codice che nel primo caso scrive la trasformazione dei vertici attraverso la trasformazione di un vertex generico. Nello fragment shader scriverà un semplice codice che determinerà il colore. Dobbiamo però definire come i vari vertici sono collegati tra loro per ottenere l'immagine finale. Noi definiamo i dati dei vertici nella host app,



generalmente in un array. I dati devono essere compresi nel sistema di coordinate di default di opengl, ovvero tra -1 e 1.

Tutti i dati devono essere copiati dalla memoria CPU alla memoria del device GPU, per far ciò si utilizza un **vertex buffer object (VBO)**. Si definisce un handle e si chiede a OpenGL di inizializzarlo. Esso gli assegna il primo unsigned int libero per effettuare tale handle. Un'altra operazione da fare è quella di collegare al VBO un target nel contesto OpenGL, ovvero un insieme di variabili di stato che vogliamo modificare. La copia dei dati viene effettuata attraverso `glBufferData`, la quale copia i dati presenti nell'array ricevuto in input.

Essenzialmente il VBO è un handle che si riferisce ai vertex data presenti nella memoria del device. VBO contiene l'indirizzo dei dati nella GPU memory perché se vogliamo di nuovo utilizzare di nuovo tali dati possiamo semplicemente utilizzare VBO piuttosto che copiare di nuovo i dati nella GPU e sprecare risorse. L'ultimo parametro definisce come i dati saranno usati. `STATIC` indica che tali dati non cambieranno, `DYNAMIC` e `STREAM` invece che cambieranno durante l'applicazione. Con dynamic OpenGL ogni volta, se i dati sono cambiati effettua una copia. Con stream invece possiamo definire che ad ogni passo OpenGL effettui la copia senza controllare.

VERTEX SHADER CODE- inizia con la dichiarazione della versione utilizzata. Ogni istanza riceve il proprio set di dati relativo al proprio vertice. I vertici in questo caso devono essere trasformati in un vertice con 4 parametri per via della regola matematiche su cui si basa OpenGL. Adesso i fragment vengono generati, tanti fragment dal processo di rasterization.